- Araştırma Ödevi -

a) Big ou, omega, theta, little ou, fonksiyonlar

Asimptotik Gösterimler: Big-O, Big-Ω, Big Θ, little-o

Algoritmaların giriş boyutu n arttıkça çalışma süresi / bellek kullanımının büyüme hızını ifade eder.

* $f(n) = O(g(n))$

Big-O (üst sınır): f(n), en fazla g(n) kadar büyür.

Tanım: $\exists c, n_0 > 0$ öyle ki $0 \leq f(n) \leq c\,g(n), n \geq n_0$

Örnek: $3n^2 + 5n + 2 = O(n^2)$

* $f(n) = \Omega(g(n))$

Big-Ω (alt sınır): f(n), en az g(n) kadar büyür.

Tanım: $\exists c, n_0 > 0$ öyle ki $f(n) \geq c\,g(n), n \geq n_0$

Örnek: $3n^2 + 5n + 2 = \Omega(n^2)$

* $f(n) = \Theta(g(n))$

Big-Θ (sıkı sınır): Hem O hem Ω ise aynı mertebededir.

Örnek: $3n^2 + 5n + 2 = \Theta(n^2)$

* $f(n) = o(g(n))$

little-o (kesin olarak küçük): f(n), g(n)'den daha yavaş büyür.

Tanım: $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

Örnek: $n = o(n^2)$, $\log n = o(n)$

Büyüme sırası:

$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$

b) Telescoping, Master Teorem

- Telescoping (Açılarak Sadeleşen Recurrence): Tekrarlı bağlantı açılır, terimler birbirini götürür.

Örnek: $T(n) = T(n-1) + n$

Açılım: $T(n) = T(n-2) + (n-1) + n$

$T(n) = T(1) + 2 + 3 + \dots + n$

Toplam: $= O(n^2)$

Geometrik örnek:

$T(n) = T(n/2) + 1$

$= 1 + 1 + 1 + \dots$ ($\log n$ kez)

Sonuç $= O(\log n)$

- Master Theorem

Biçim: $T(n) = aT(n/b) + f(n)$

- a: alt problem sayısı
- b: boyut küçülme oranı
- f(n): bölme / birleştirme maliyeti

Karşılaştırma: $n^{\log_b a}$

Case 1:

$f(n) = O(n^{\log_b a - \varepsilon})$

$\rightarrow T(n) = \Theta(n^{\log_b a})$

Case 2:

$f(n) = \Theta(n^{\log_b a} \log^k n)$

$\rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Case 3:

$f(n) = \Omega(n^{\log_b a + \varepsilon})$

ve regularity varsa

$\rightarrow T(n) = \Theta(f(n))$

Örnekler

- Merge Sort: $T(n) = 2T(n/2) + n = \Theta(n \log n)$
- Binary Search: $T(n) = T(n/2) + 1 = \Theta(\log n)$
- $T(n) = 4T(n/2) + n^2 = \Theta(n^2 \log n)$

c) ADT'ler, Pointer Based / Array Based Yapılar, Amortized Cost

ADT (Abstract Data Type)

Veri yapısının ne yaptığı önemlidir, nasıl tuttuğu değil.
Örn; Stack, Queue, List, Deque.

List

- insert(i, x)
- delete(i)
- get(i)

Array-based List

- Bellekte ardışık tutulur.
- index erişimi = O(1)
- araya ekleme / silme = kaydırma gerekir O(n)

Pointer-based (Linked List)

- Node + next pointer
- başa ekleme / silme O(1)
- rastgele erişim O(n)

Stack (LIFO)

Son giren ilk çıkar.

- push
- pop
- top

array stack: üst indeks tutulur.
push / pop = O(1)

linked stack: head üzerinden yapılır.
push / pop = O(1)

Queue (FIFO)

İlk giren ilk çıkar.

- enqueue
- dequeue

Circular array queue: front / rear ile O(1)

Linked queue: head / tail ile O(1)

Deque (Double Ended Queue)

İki uçtan ekleme - silme:

- addFirst / addLast
- removeFirst / removeLast

Doubly linked list veya circular array ile verimli.

Amortized Cost

Tek işlem pahalı olabilir ama çok işlem ortalaması ucuzdur.
Örnek dynamic array: kapasite dolunca 2 katına çıkarılır. Bazı eklemeler O(n), çoğu O(1).
Ortalama append maliyeti = O(1)

d) Bubble, Selection, Insertion Sort

Bubble Sort

Yan yana elemanları karşılaştırır, büyük olanı sağa iter. Geçişler sonunda en büyük sona gider.

- Best: O(n) (optimized)
- Avg / Worst: $O(n^2)$
- Stable: Evet
- In-place: Evet

Selection Sort

Her turda minimum bulunur, başa alınır.

- Best / Avg / Worst: $O(n^2)$
- Swap sayısı az
- Stable: Hayır (genelde)
- In-place: Evet

Insertion Sort

Kart dizer gibi sıralı kısmın içine yerleştirir.

- Best (zaten sıralı): O(n)
- Avg / Worst: $O(n^2)$
- Stable: Evet
- In-place: Evet

* Küçük veri seti: Insertion iyi
* Swap pahalıysa: Selection
* Eğitim amaçlı basitlik: Bubble
* Büyük veri için genelde Merge / Quick / Heap tercih edilir.